Neural Network Generative Models: from GANs to Diffusion with Text

Lecture 20
Notes for Probabilistic Models of the Visual Cortex

Alan Yuille

Johns Hopkins University

10 November, 2026

What this lecture covers

Deep generative models

Generative adversarial networks

StyleGAN

GAN inversion

Autoencoders

Diffusion with text

Summary

Sources, figures and colors

The lecture follows two sets of slides by Alan Yuille: one on generative adversarial networks (GANs), StyleGAN and autoencoders (prepared with Wei Chen), and one on diffusion models conditioned on text. The formulas the slides state or ask for come from the papers cited next to them: Goodfellow et al. (2014) for GANs, Kingma and Welling (2014) for variational autoencoders, Ho et al. (2020) for diffusion models and Rombach et al. (2022) for Stable Diffusion.

The diagrams are drawn again for these notes, with the photographs of the original slides. In the comparison of generative models and in the autoencoder diagram, green marks the input image and the encoder, blue the decoder (or generator) and its output, and red the latent variable, as in the figures of Weng (2018, 2021). Red is also the color of an annotation (what training uses, what is unknown). Other drawings use colors only to tell elements apart. Pictures taken from a paper say so in their caption.

Deep networks learn to generate images from big data

A *generative model* of images is a probability distribution from which new images can be drawn. A photometric model renders an explicit scene (3D shape, material, lighting), so it generalizes to new viewing conditions. A deep network learns from big data: a *generator* maps a latent z, a vector of random numbers from a simple distribution such as a Gaussian, to an image. Its latent variables have no fixed physical meaning, which makes it hard to interpret.

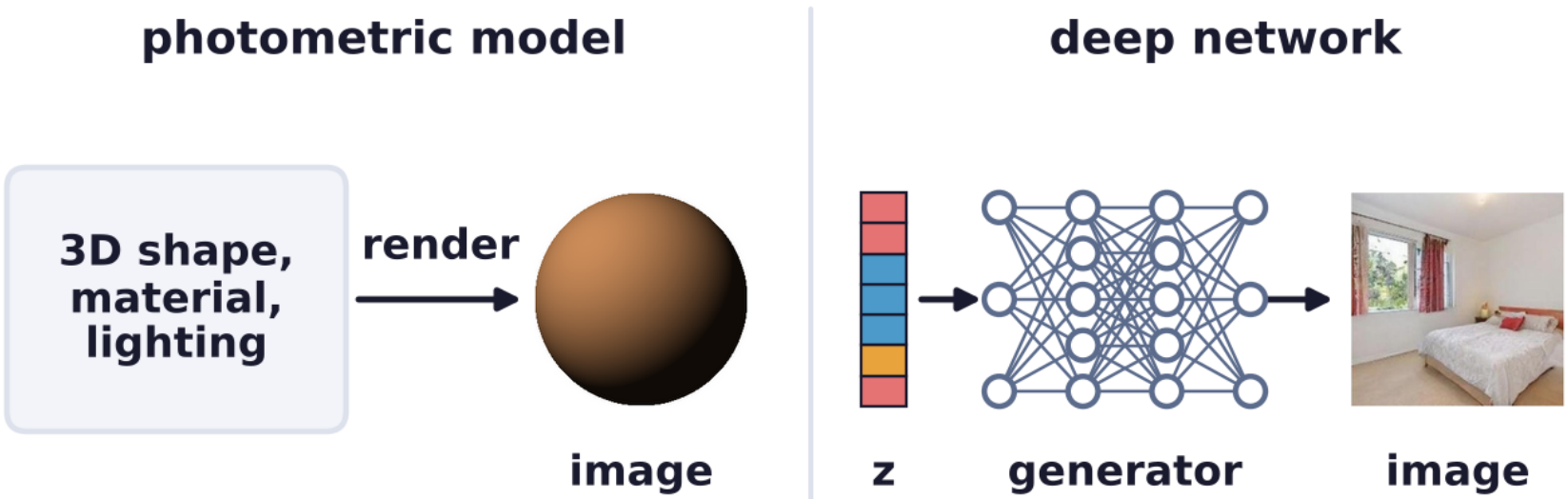


Figure 1: Two routes to an image: a scene description rendered, and a latent vector z (colored cells) turned into an image by a learned generator.

GANs improved the generated images year by year

The approach that first made deep image generation work well was the generative adversarial network (GAN) of Goodfellow et al. (2014), whose first faces were small and blurred. Within six years the quality and complexity of the images grew to photographs that are hard to tell from real ones:

- DCGAN, a deep convolutional GAN (Radford et al., 2016), built the generator from convolution layers;
- PG-GAN, progressive growing of GANs (Karras et al., 2018), trains at low resolution first and adds layers for finer detail;
- StyleGAN, a style-based generator (Karras et al., 2019), and its second version StyleGANv2 (Karras et al., 2020a) feed latent variables to every level of the network;
- BigGAN (Brock et al., 2019) trains a large GAN on many object categories;
- StyleGAN-ADA, StyleGAN2 trained with adaptive discriminator augmentation (Karras et al., 2020b), learns from a few thousand images, such as the cats and the medical images of its column.

GANs improved the generated images year by year

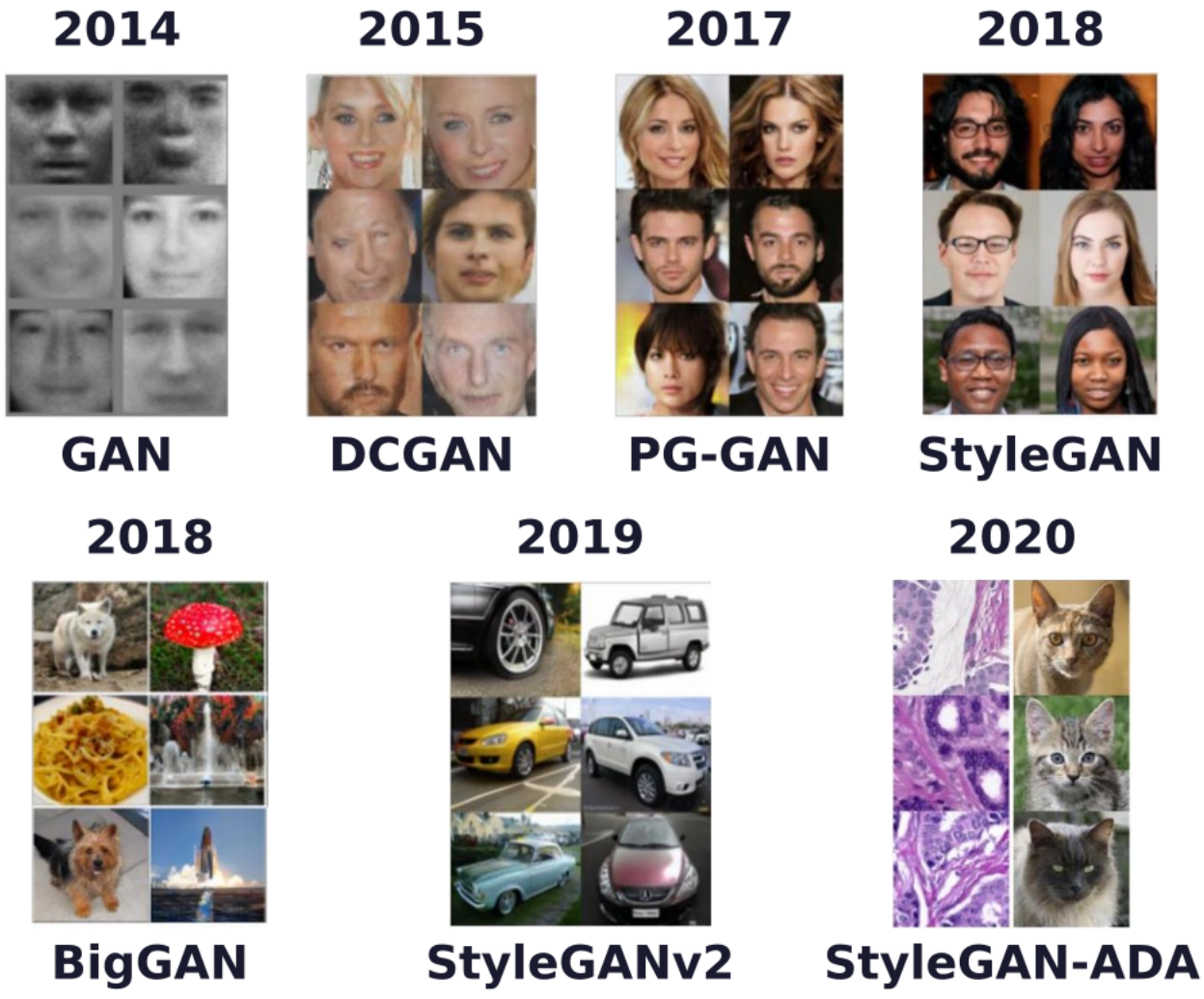


Figure 2: Images generated by seven GANs, by year of the model's first preprint (from the original slides; years and names set again). The first four show faces; BigGAN generates many object categories, StyleGANv2 cars, StyleGAN-ADA histology images and cats.

NeRF generates highly realistic new views (2020)

A neural radiance field (NeRF; Mildenhall et al., 2020) represents one scene. A multilayer perceptron (MLP, a fully connected network) maps a 3D position and a viewing direction to a color and a density, and an image is computed by integrating colors along the ray of each pixel, as light travels through a semi-transparent medium. The network is trained on many photographs of the scene taken from known cameras, until the images it computes for those cameras match the photographs.

The trained network then renders the scene from new viewpoints, with highly realistic results. NeRF stands between the two routes: the geometry of cameras, rays and volume rendering is built in, while the density and the color are learned from data and stored in the weights of the network.

Diffusion models generate realistic images prompted by text

Diffusion models generate highly realistic images, and they can be prompted by text. The prompt "An astronaut riding a horse in photorealistic style" is an example of OpenAI's DALL-E 2 (OpenAI, 2022). Generating images from a sentence was a breakthrough: the text says what the image should contain, including combinations that no training image shows.

A diffusion model is a special case of an *autoencoder*, a network that encodes its input into a latent representation and reconstructs the input from it.

Text-prompted diffusion models are a big industry. The slide names a few generative systems: OpenAI's DALL-E 2, Google's Imagen (Saharia et al., 2022), Meta's Make-A-Video for videos, and OpenAI's large language model GPT-4.

A GAN is a game between a generator and a discriminator

A GAN consists of two networks that play a game (Goodfellow et al., 2014).

- The *generator* G takes a latent z, drawn from a fixed distribution p_z, and outputs an image $G(z)$. It is trained to generate images similar to the training data.
- The *discriminator* D takes an image x and outputs $D(x) \in [0, 1]$, the probability that x is real, that is, a training image. It is trained to predict whether an image is real or generated.

The two goals conflict: D improves by catching the generated images, and G improves by making images that D takes for real ones.

The generator defines a distribution p_g over images, the distribution of $G(z)$ when $z \sim p_z$, without a formula for its density: images can be sampled from it, but their probability cannot be evaluated. GANs are therefore called implicit generative models.

A GAN is a game between a generator and a discriminator

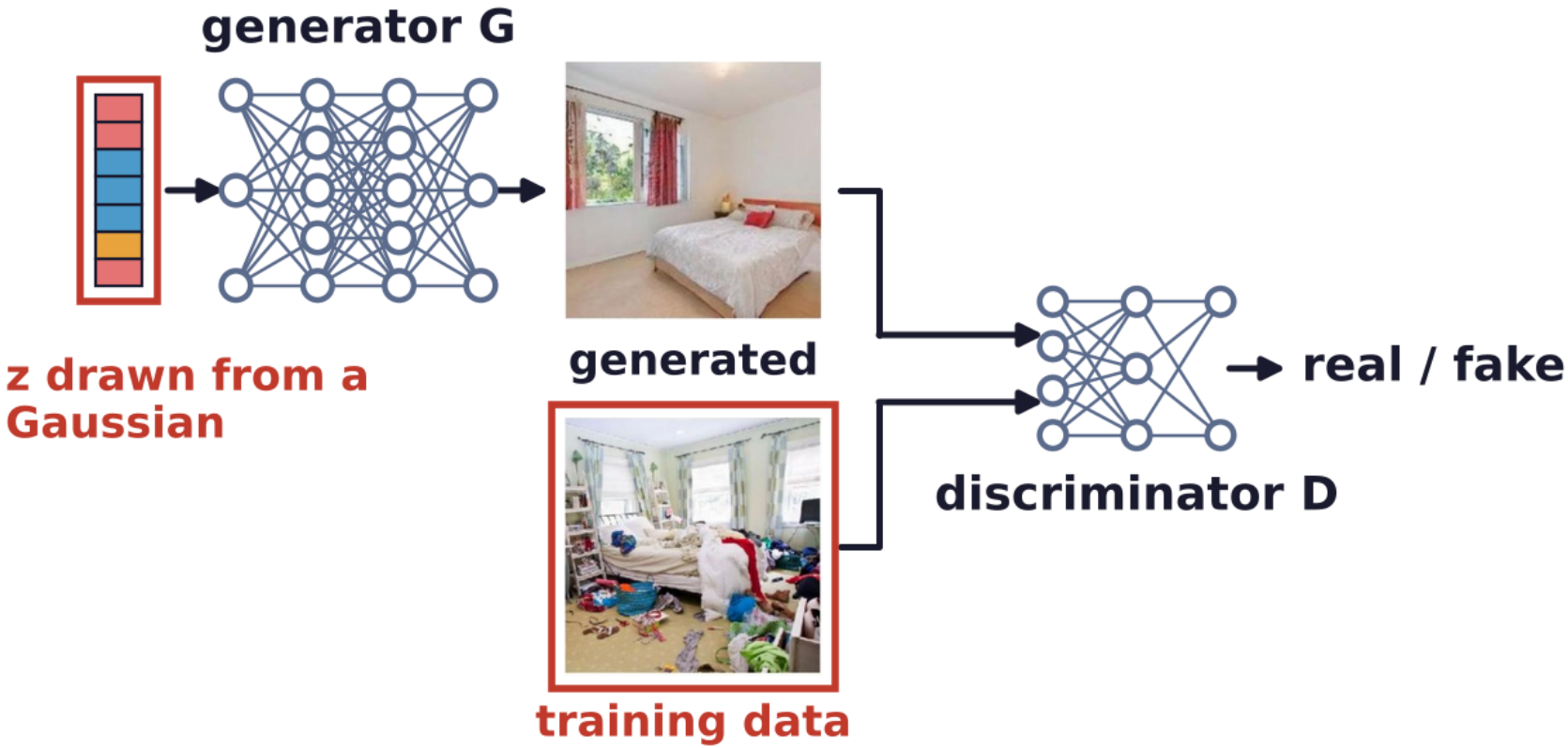


Figure 3: The GAN of the original slides, drawn again with their bedroom photographs. The generator turns z into a generated (fake) bedroom; the discriminator sees generated and real images and outputs real or fake. Red marks what training uses: the training data and the distribution of z.

The generator is a counterfeiter, the discriminator the police

Goodfellow (2016) explains the game with an analogy. The generator is a counterfeiter who prints fake money; the discriminator is the police, who try to tell fake money from real money. Each side improves against the other, and the competition ends when the counterfeits cannot be told from the real thing.

The generator is a structured probabilistic model with latent variables z as input and an image x as output. The discriminator is a conventional deep network, a classifier with an image as input and a binary decision, real or fake: $D(x)$ is the probability of "real".

Both are *differentiable functions*. This is what makes the game trainable: the gradient of D's output with respect to its input image flows back through $G(z)$ into the parameters of the generator, and tells the generator how to change its images so that D takes them for real.

The generator is a counterfeiter, the discriminator the police

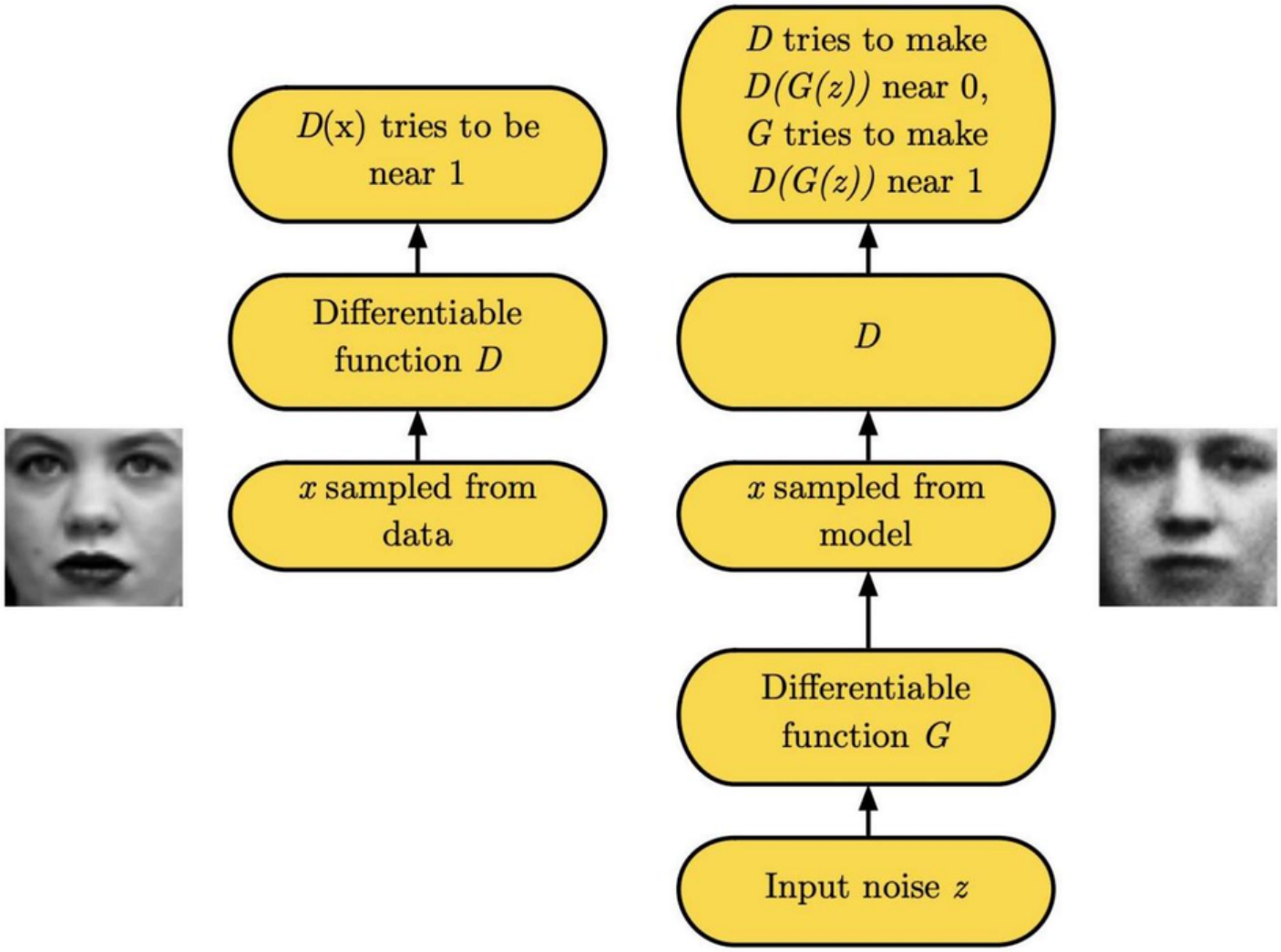


Figure 4: Goodfellow (2016), Figure 12. Left path: a training image goes through D, which tries to output near 1. Right path: noise z goes through G, and its image through the same D, which tries to output near 0 while G tries to make it output near 1.

Training uses real images and latents from a Gaussian

Training needs two kinds of samples.

- *Training data*: a set of real images $x^{(1)}, \ldots, x^{(n)}$, for instance photographs of bedrooms. They are samples of the data distribution p_{data}, which is unknown.
- *Latents*: vectors z drawn from a pre-defined distribution p_z, typically a standard Gaussian $\mathcal{N}(\mathbf{0}, \mathbf{I})$ (DCGAN used a uniform distribution) with a hundred to a few hundred dimensions (512 for StyleGAN). The distribution is fixed before training and never learned.

Each latent gives one generated image $G(z)$. The generator never sees a training image directly: it learns about the training data only through the gradients that the discriminator passes back to it.

The two kinds of samples enter as minibatches: m training images and m latents per step.

Training alternates discriminator and generator updates

Goodfellow et al. (2014, Algorithm 1) train the two networks in turn. In each iteration:

1. *Discriminator update* (k times, with G held fixed): sample m latents $z^{(i)}$ and m training images $x^{(i)}$, and take a step of gradient ascent on

$$\frac{1}{m}\sum_{i=1}^{m}\left[\log D\big(x^{(i)}\big) + \log\Big(1 - D\big(G\big(z^{(i)}\big)\big)\Big)\right].$$

   D learns to call training images real and generated ones fake.

2. *Generator update* (with D held fixed): sample m new latents and take a step of gradient descent on $\frac{1}{m}\sum_i \log\big(1 - D(G(z^{(i)}))\big)$. Only generated images enter, because the real images do not depend on G. G learns to make D call its images real.

Goodfellow et al. use $k = 1$.

At convergence the generator alone makes realistic images

When training has converged, the discriminator is set aside. To make a new image, draw a new latent $z \sim p_z$ and compute $G(z)$: one forward pass of the generator.

Ideally the generated images then follow the same distribution as the training images, $p_g = p_{\text{data}}$. The discriminator can then do no better than guess: the best discriminator outputs $1/2$ everywhere.

In practice convergence is approximate, and it is judged by looking at samples and by comparing statistics of generated and real images. The generator is a continuous function of z, so nearby latents give similar images, and moving z along a line changes the image gradually.

GAN training is a minimax game

Goodfellow et al. (2014, eq. 1) write the game with one value function:

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}(x)}\big[\log D(x)\big] + \mathbb{E}_{z\sim p_z(z)}\big[\log\big(1-D(G(z))\big)\big].$$

Here $\mathbb{E}$ is the expectation, p_{data} the distribution of the training images and p_z the distribution of the latent.

- D maximizes V: $\log D(x)$ is large when real images get $D(x)$ near 1, and $\log(1 - D(G(z)))$ is large when generated images get $D(G(z))$ near 0.
- G appears only in the second term and minimizes V: it wants $D(G(z))$ near 1, so that its images are taken for real ones.

A *minimax* game is one in which one player minimizes what the other maximizes; the alternating updates are gradient steps for the two players.

GAN training is a minimax game

In practice the generator maximizes $\log D(G(z))$ instead of minimizing $\log(1 - D(G(z)))$. Both push $D(G(z))$ toward 1, but they give different gradients early in training, when D rejects the generated images easily.

Write the discriminator's output as $D = \sigma(a)$, with a its logit and $\sigma(a) = 1/(1 + e^{-a})$. A confident rejection is a large negative a. The derivatives with respect to a are

$$\frac{\partial}{\partial a} \log\big(1 - \sigma(a)\big) = -\sigma(a), \qquad \frac{\partial}{\partial a} \log \sigma(a) = 1 - \sigma(a).$$

As $a \to -\infty$ the first tends to 0 and the second to 1. The original loss saturates exactly where the generator is worst, while the second gives a gradient of full size. Goodfellow et al. call it the stronger gradient early in learning. The two players then optimize different objectives, with the same fixed point.

The optimal discriminator compares the two densities

Fix G. Writing the second expectation over the generated images $x = G(z)$, whose density is p_g, gives

$$V(D, G) = \int \Big[p_{\text{data}}(x) \log D(x) + p_g(x) \log \big(1 - D(x)\big) \Big] \, dx.$$

The integrand can be maximized at each x separately. For $a, b > 0$ the function $y \mapsto a \log y + b \log(1 - y)$ on $(0, 1)$ has derivative $a/y - b/(1 - y)$, which vanishes at $y = a/(a + b)$, its maximum. Hence (Goodfellow et al., Proposition 1)

$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.$$

The best discriminator is the ratio of the data density to the sum of both densities: near 1 where only real images occur, near 0 where only generated ones occur, and $1/2$ where the two densities are equal.

The optimal discriminator compares the two densities

Put D_G^* back into V and call the result $C(G) = \max_D V(D, G)$. With the average density $\bar{p} = (p_{\text{data}} + p_g)/2$,

$$\begin{aligned} C(G) &= \mathbb{E}_{p_{\text{data}}}\left[\log \frac{p_{\text{data}}}{2\bar{p}}\right] + \mathbb{E}_{p_g}\left[\log \frac{p_g}{2\bar{p}}\right] \\ &= -\log 4 + D_{\text{KL}}(p_{\text{data}} \,\|\, \bar{p}) + D_{\text{KL}}(p_g \,\|\, \bar{p}), \end{aligned}$$

where D_{KL} is the Kullback–Leibler divergence. The two KL terms together are twice the Jensen–Shannon divergence (JSD): $C(G) = -\log 4 + 2\,\text{JSD}(p_{\text{data}} \,\|\, p_g)$ (Theorem 1). The JSD is at least 0, and 0 only when the two distributions are equal, so the global minimum of $C(G)$ is $-\log 4$, reached at $p_g = p_{\text{data}}$, where $D_G^* = 1/2$ everywhere.

The analysis assumes that D is optimal at every step. Training takes a few gradient steps on each player instead, and the minimax game is hard to optimize: the two networks can oscillate around each other, or the generator can collapse to a few images that currently fool D (mode collapse; Goodfellow, 2016).

StyleGAN has latent variables at every level of the network

A traditional GAN takes input from a single latent variable z, fed to the first layer. StyleGAN (Karras et al., 2019) has latent variables at every level of the network. Its generator has two parts.

- The *mapping network* F normalizes $z \in \mathcal{Z}$ (512 numbers) and passes it through eight fully connected layers, giving an intermediate latent $w \in \mathcal{W}$ (also 512 numbers). The paper writes f; the slides write F.
- The *synthesis network* starts from a learned constant tensor of 4×4 positions with 512 channels, and alternates 3×3 convolutions, upsampling to 8×8, 16×16, up to 1024×1024, and adaptive instance normalization (AdaIN) operations.

At every AdaIN, a learned affine map A (one per layer) turns w into a *style* $y = (y_s, y_b)$, and a block B adds Gaussian noise, scaled by learned per-channel factors, to the features just before. The styles set the global appearance; the noise sets stochastic detail such as the exact placement of hairs and freckles.

StyleGAN has latent variables at every level of the network

AdaIN acts on each feature map x_i (channel i; here x_i is a feature map, not an image) separately:

$$\mathrm{AdaIN}(x_i, y) = y_{s,i} \frac{x_i - \mu(x_i)}{\sigma(x_i)} + y_{b,i},$$

where $\mu(x_i)$ and $\sigma(x_i)$ are the mean and the standard deviation of the feature map over its spatial positions. The map is normalized to mean 0 and standard deviation 1, then scaled by $y_{s,i}$ and shifted by $y_{b,i}$. Each AdaIN therefore overrides the statistics of the features that reach it, and the style of each level controls that level only.

The levels divide the work (Karras et al., 2019, Sec. 3.1): styles at the coarse resolutions (4^2 to 8^2) set pose, face shape and eyeglasses; the middle ones (16^2 to 32^2) facial features and hair style; the fine ones (64^2 to 1024^2) the color scheme and fine structure. Taking the coarse styles from one latent and the fine ones from another mixes the two images in this way.

StyleGAN has latent variables at every level of the network

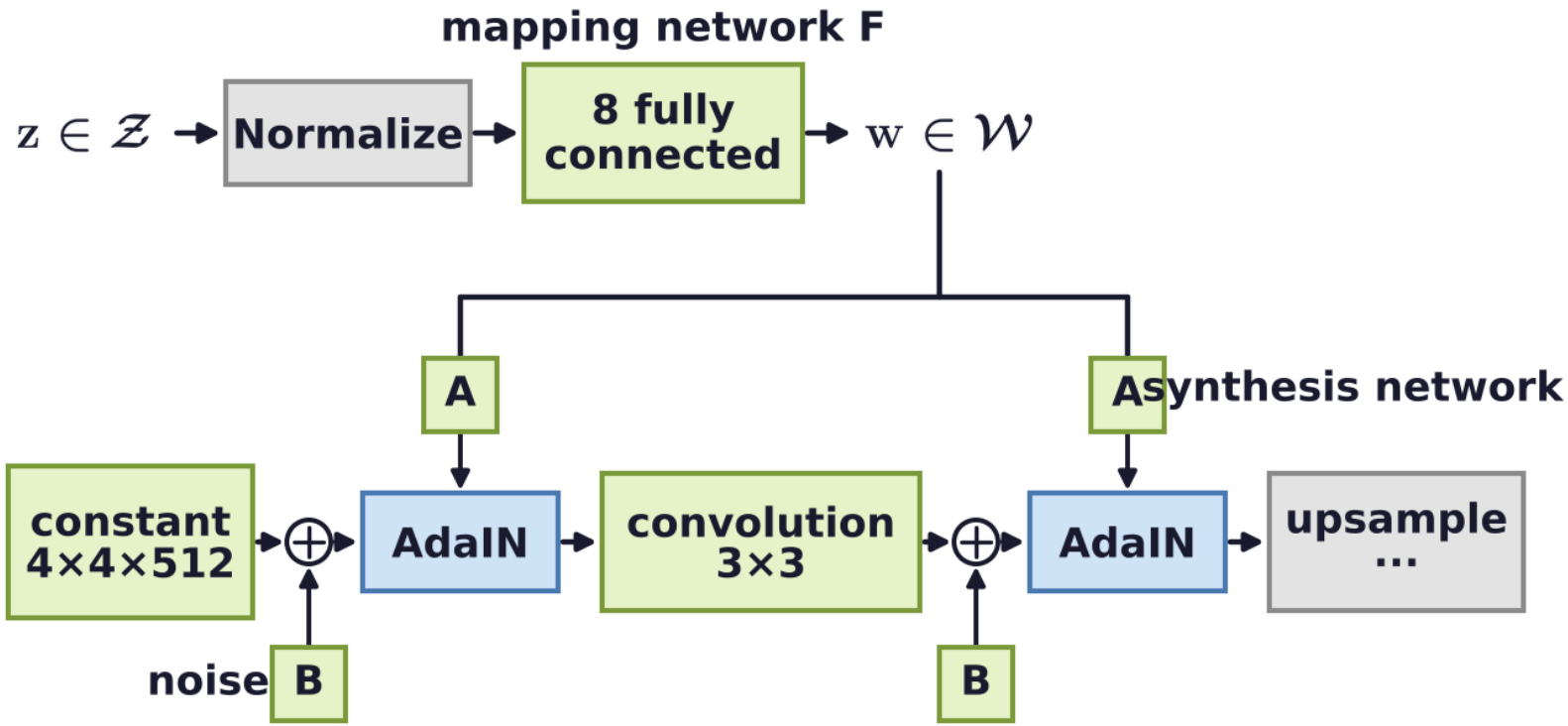


Figure 5: The style-based generator of Karras et al. (2019, Figure 1b), drawn again (4×4 level only). Grey: fixed operations; green: learned layers; blue: AdaIN; $\oplus$: addition of the noise B.

A mapping network F turns z into an intermediate latent w

StyleGAN embeds the input latent code z into an intermediate latent space, $w = F(z)$; now w controls the style, that is, the factors of variation of the domain (for faces, masculinity and hair length). The mapping can "unwarp" z so that the factors become more linear: z follows a fixed Gaussian, so if a combination never occurs in training, say long-haired males, the map from $\mathcal{Z}$ must bend around the gap; the learned $\mathcal{W}$ can take the shape of the data.

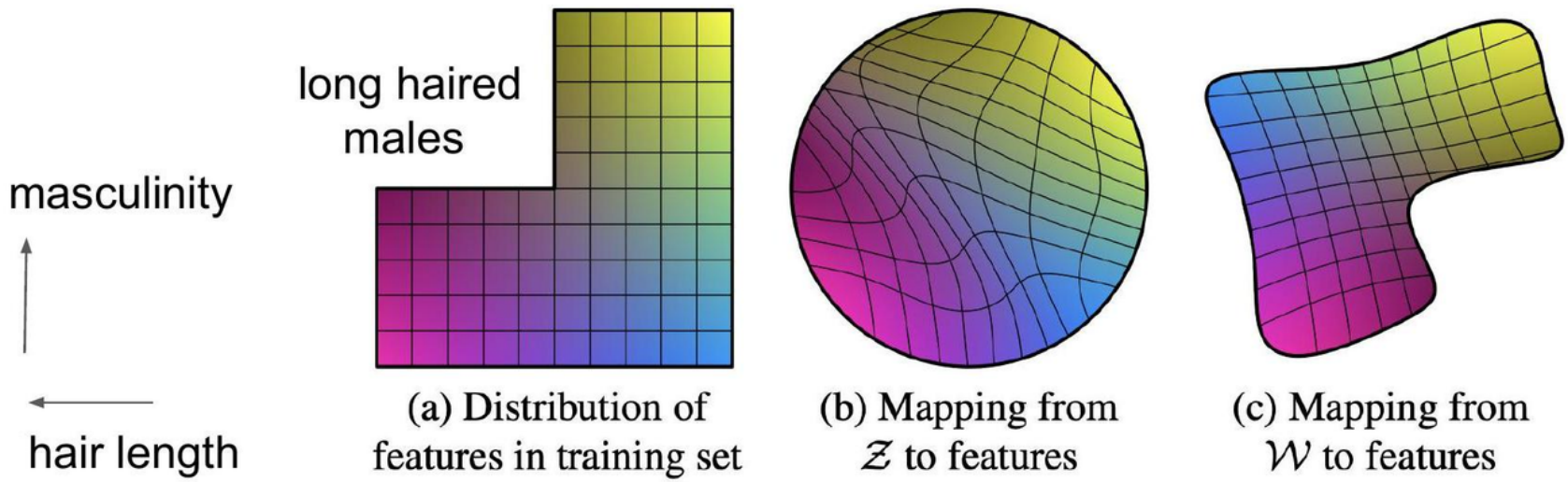


Figure 6: Karras et al. (2019), Figure 6: (a) the factors in the training set, with long-haired males missing; (b) the warped mapping from $\mathcal{Z}$; (c) the mapping from $\mathcal{W}$.

A disentangled latent space has one direction per factor

There are various definitions of disentanglement. A common goal is a latent space that consists of linear subspaces, each of which controls one factor of variation: moving along one direction changes the smile and nothing else, along another the gaze, and so on.

Wu et al. (2021) look inside StyleGAN for such directions. The style space $\mathcal{S}$ is the set of all style parameters y of all layers. They take a real photograph, find by GAN inversion the latent code whose generated image reproduces it (the "inverted" image), and then change a single channel of the style.

Each change acts on one local attribute: a smile, lipstick, the gaze, the eye shape, frowning eyebrows, a goatee, a bulbous nose. The rest of the face stays as it was, which is the sign of a disentangled representation.

A disentangled latent space has one direction per factor

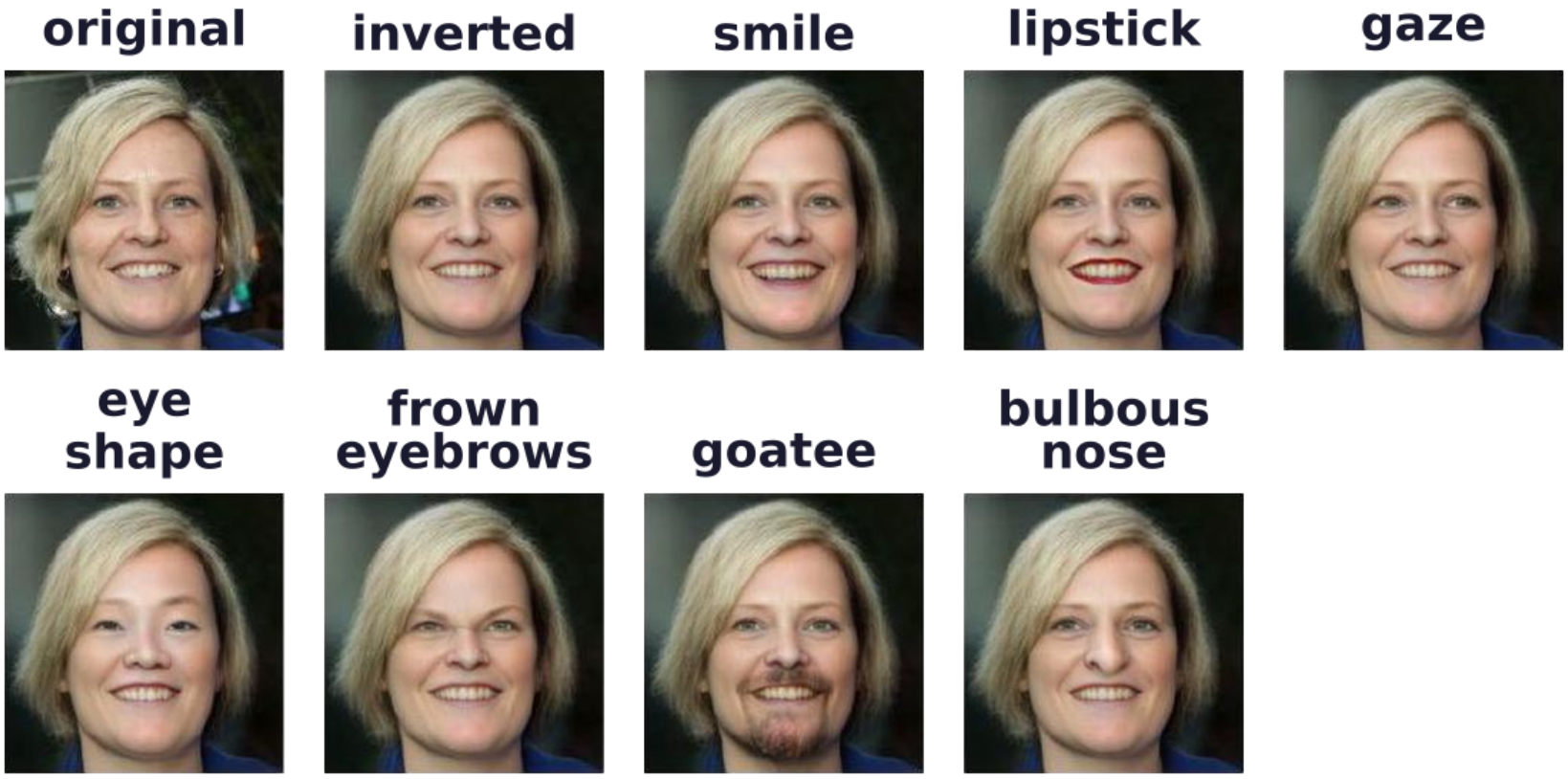


Figure 7: Wu et al. (2021), Figure 1, first person: the photograph, its inversion, and seven edits, each changing one channel of the style space (labels set again).

The latent spaces Z, W and S are increasingly disentangled

StyleGAN has three latent spaces, one after the other.

- $\mathcal{Z}$: the input latent, $z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
- $\mathcal{W}$: $w = F(z)$, the output of the mapping network, a multilayer perceptron (MLP).
- $\mathcal{S}$: the layer-wise style codes $y_i = A_i(w)$, the inputs of the AdaIN operations (in StyleGAN2, the scales that modulate the convolution weights).

Which one is the most disentangled? Karras et al. (2019, Sec. 4) measure two things: how smoothly the image changes when the latent moves along a line (perceptual path length), and how well attributes such as gender or glasses are separated by a hyperplane in the latent space (linear separability). $\mathcal{W}$ scores better than $\mathcal{Z}$ on both. Wu et al. (2021) find the style space $\mathcal{S}$ "significantly more disentangled" than the earlier spaces, $\mathcal{W}$ included.

The latent spaces Z, W and S are increasingly disentangled

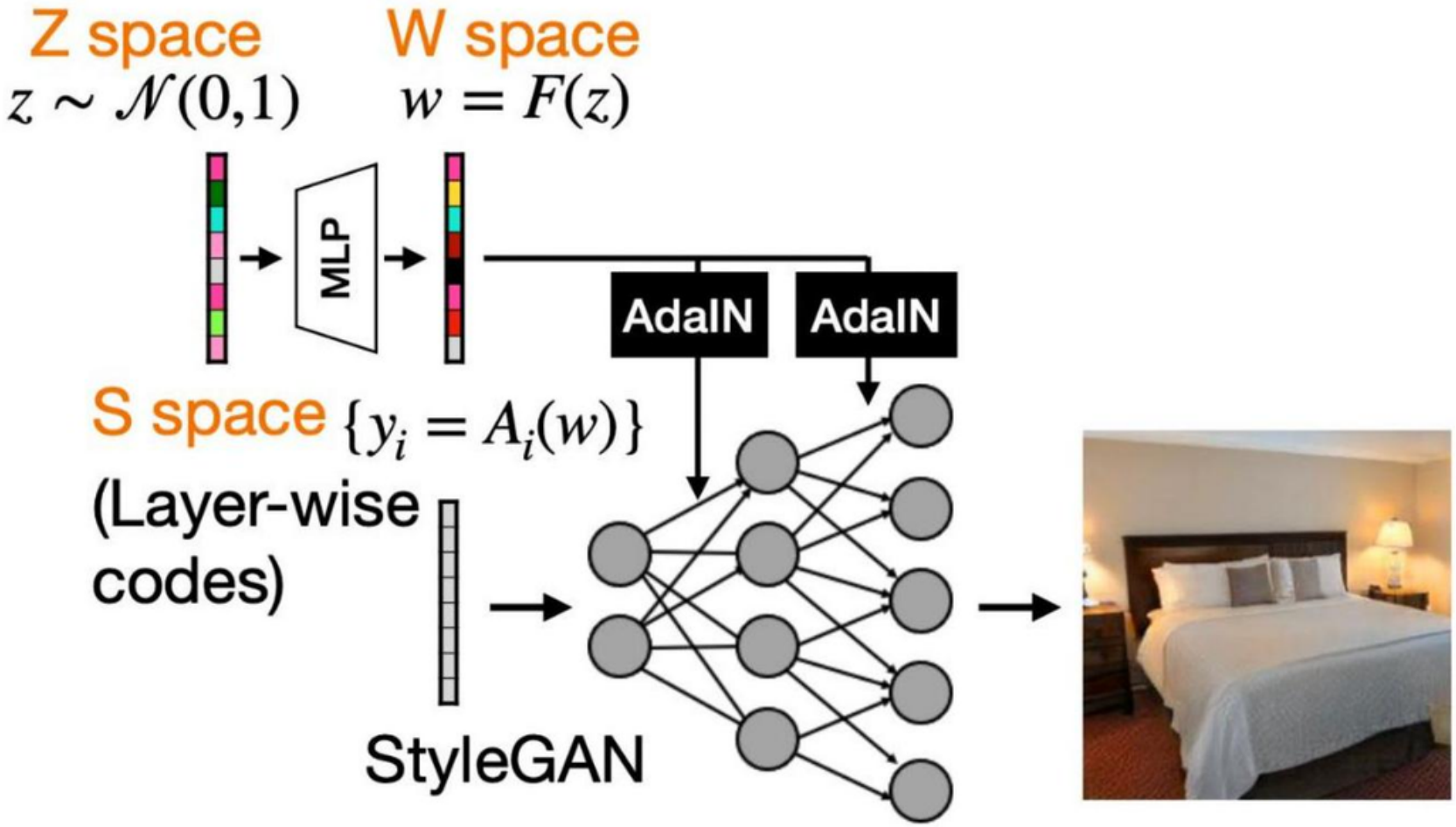


Figure 8: The three latent spaces of StyleGAN (from Wu et al., 2021): z from a Gaussian, $w = F(z)$ from the mapping network (MLP), and the styles $y_i = A_i(w)$ that enter the AdaIN operations.

GAN inversion finds the latent code of a target image

A pretrained GAN can be applied to image processing. Its generator knows what images of its domain look like, and GAN inversion uses that knowledge. Given a target image I, inversion finds the latent code whose generated image reconstructs it:

$$x^* = \arg\min_x \left\| G(x) - I \right\|.$$

Here x is the latent code, as in the original slides (for GANs, x denoted an image), G the pretrained generator and I the target image.

G is fixed and differentiable, so x^* is found by gradient descent on x, starting for instance from the average latent. The distance is often measured on the features of a pretrained network as well as on the pixels. Gu et al. (2020) use several latent codes at once, combined inside the generator, to reconstruct real images more faithfully.

GAN inversion finds the latent code of a target image

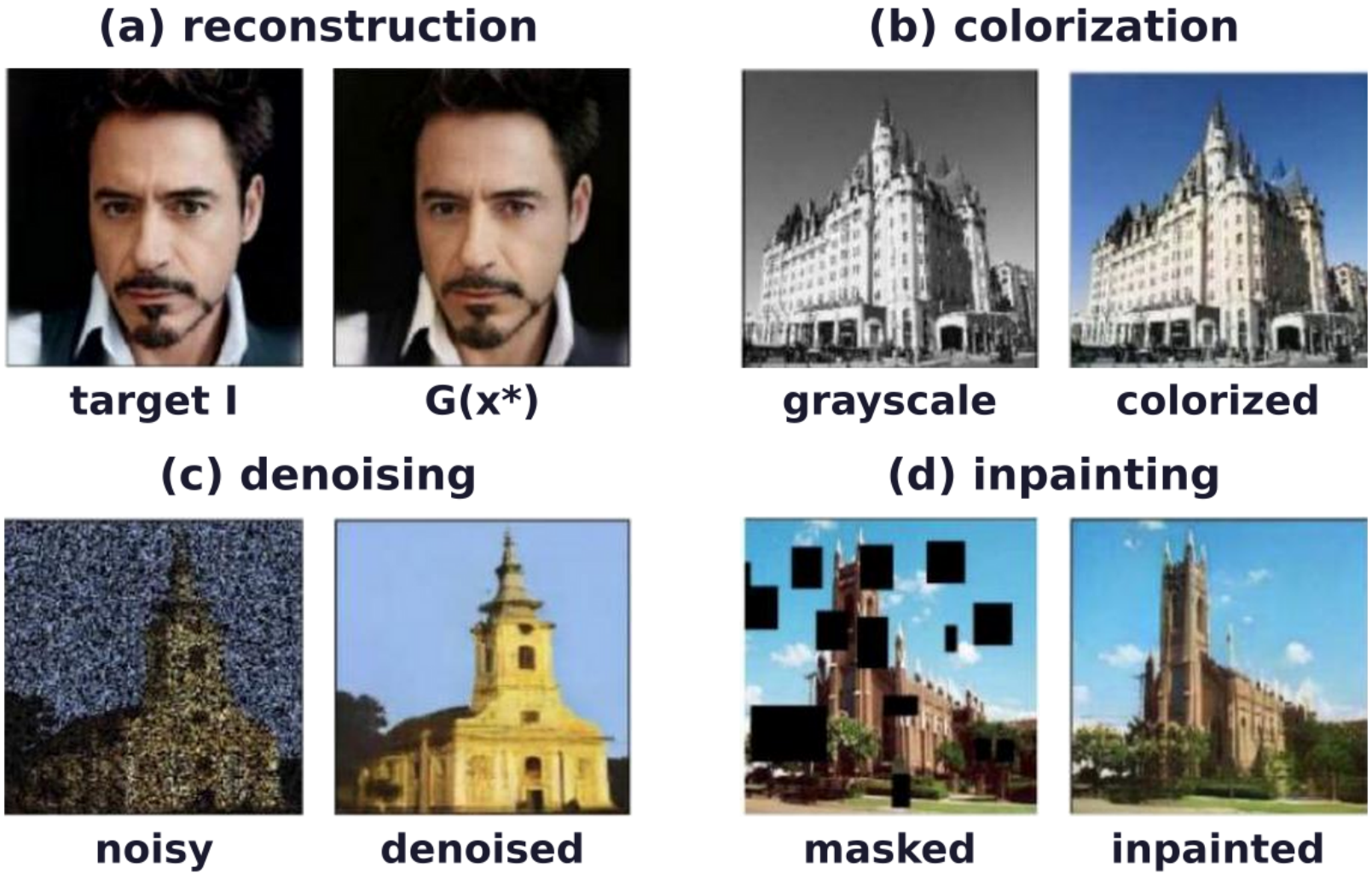


Figure 9: GAN inversion and its applications (Gu et al., 2020, from the original slides): (a) reconstruction of a target image; (b) colorization; (c) denoising; (d) inpainting of masked regions.

Inversion colorizes and super-resolves images

The same optimization solves other problems once the observed image is written as a known degradation H of a clean image:
$x^* = \arg\min_x \|H(G(x)) - I_{\text{obs}}\|$, and the answer is $G(x^*)$.

- *Colorization*: find the latent x^* whose generated image, made gray, matches the grayscale target:
  $x^* = \arg\min_x \|\text{rgb2gray}(G(x)) - I_{\text{gray}}\|$. The generator supplies the colors, which the target does not contain.
- *Super-resolution*: find the latent whose generated image, downsampled, matches the low-resolution target:
  $x^* = \arg\min_x \|\text{down}(G(x)) - I_{\text{small}}\|$. The generator supplies the fine detail.

In Bayesian terms, the degradation gives the likelihood of the observation, and the generator restricts the answer to images the GAN can produce, which acts as the prior. The original slides show no super-resolution example.

Inversion denoises and inpaints images

Denoising is the same optimization as reconstruction, with the noisy image as target. The generator produces clean images of its domain, so the best match to a noisy image is a clean image close to it.

Inpainting fills in missing parts. A mask m has value 1 on the known pixels and 0 on the missing ones, and the dot is the product pixel by pixel:

$$x^* = \arg\min_x \left\| m \cdot G(x) - m \cdot I_{\text{context}} \right\|.$$

Only the known pixels of the context image are compared, and $G(x^*)$ supplies the missing ones, consistent with the rest of the image.

In all four applications the generator is never retrained: one pretrained model serves as the prior for many tasks.

In-domain inversion keeps the code semantically meaningful

Inversion by optimization can find a code that reproduces the pixels of the target and still lies far from the codes the generator was trained on. The reconstruction is good, but moving such a code along a semantic direction (more smile, older) gives poor images, because the code is out of the generator's domain.

In-domain GAN inversion (Zhu et al., 2020) keeps the code among the generator's own codes, so that it can be edited semantically. It trains an encoder that maps images to codes on which the generator was trained, and uses this encoder both to start the optimization and to keep the result near the encoder's answer.

One test is *semantic diffusion* (a name unrelated to diffusion models): the central face region of one person (the foreground) is pasted into the photograph of another (the context), and the inversion turns the pasted patch into a single natural face that keeps the features of the patch and adapts the rest to the context.

In-domain inversion keeps the code semantically meaningful

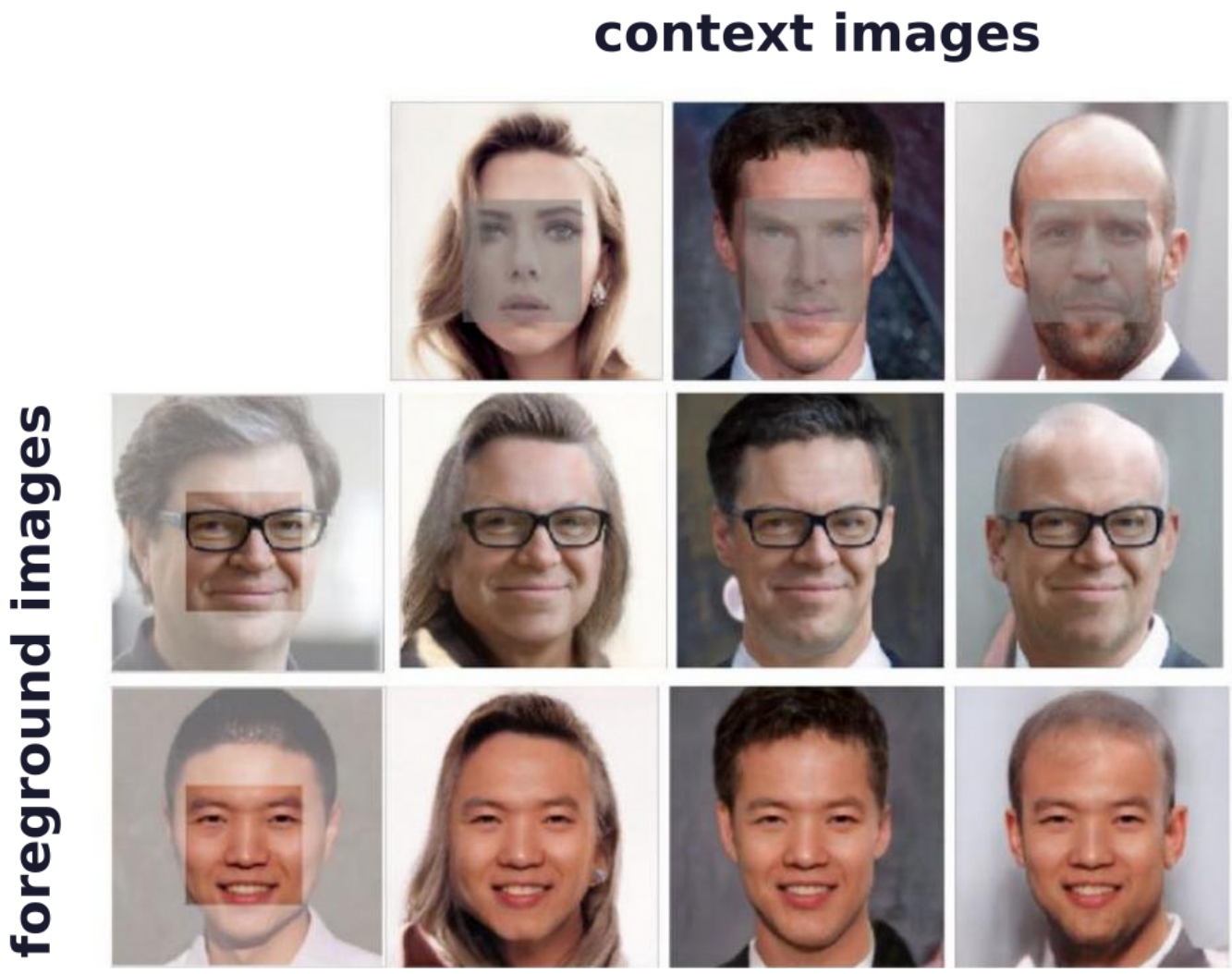


Figure 10: Semantic diffusion (Zhu et al., 2020, from the original slides). Top row: the context images, with the pasted region shown as a pale box; left column: the foreground faces. Each inner image is the result for the foreground of its row and the context of its column.

An encoder maps a real image into StyleGAN space

Optimization needs hundreds of gradient steps per image. An *encoder* E gives the latent code $x = E(I)$ of an image I in one forward pass. It is trained with G fixed, so that $I' = G(x)$ reproduces I on pixels and on the features of a pretrained network.

The encoder and the generator form an autoencoder whose decoder is the pretrained generator. The encoder is fast but less exact than optimization; the two are often combined.

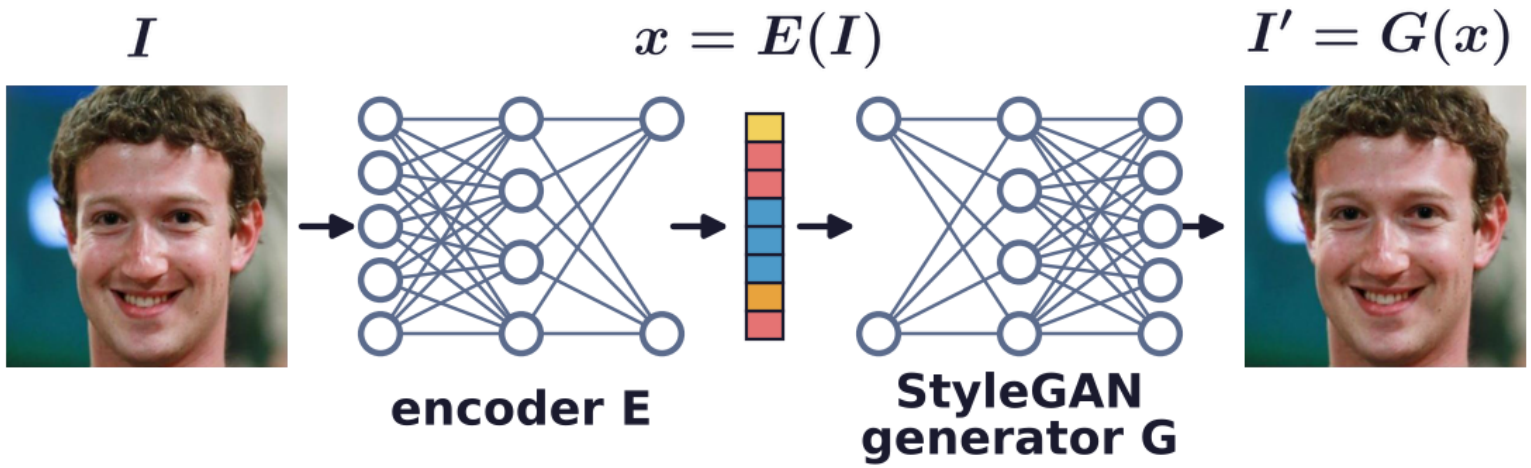


Figure 11: Encoding a real image into StyleGAN space (from the original slides).

Fidler's 3D neural rendering autoencoder uses StyleGAN

Zhang et al. (2021), from Sanja Fidler's group, combine encoding and generation in one flow, $I \mapsto W \mapsto z \mapsto I'$, which as a whole can be viewed as an autoencoder. Here W is a set of 3D scene parameters (shape, texture, lighting, camera) and F a renderer; the letters are unrelated to StyleGAN's $\mathcal{W}$ and mapping F.

- $I \to W$: an encoder $P_\phi(W \mid I)$ estimates the 3D parameters, with a differentiable renderer $I' = F(W; \alpha)$, whose parameters are α, as the decoder: the photometric route learned as an autoencoder.
- $z \to I'$: a learned StyleGAN generative model $P_\theta(I' \mid z)$ produces realistic images.
- $W \to z$: a mapping $f_\psi(z \mid W)$ links the two, learned with an autoencoder reconstruction loss: for instance, sample z, generate $I = G(z)$, estimate W and map it to z'; the loss compares z and z'.

This is feasible because the latent variables of StyleGAN are fairly interpretable, so the mapping $f_\psi(z \mid W)$ need not be very complex.

Fidler's 3D neural rendering autoencoder uses StyleGAN

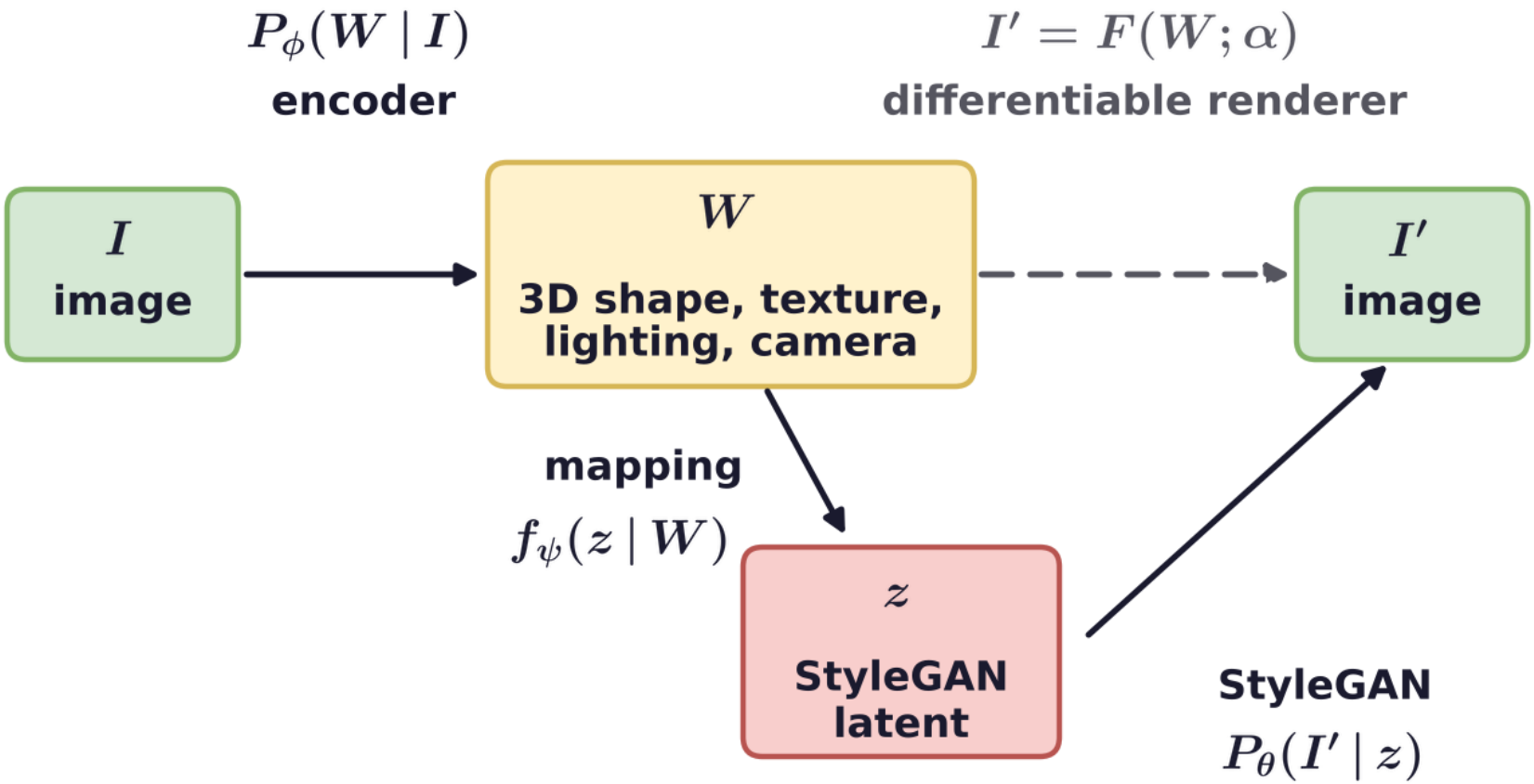


Figure 12: The flow of GANverse3D (Zhang et al., 2021), drawn for the original slides' description: the encoder to the 3D parameters W, the differentiable renderer back to an image (dashed), and the mapping to a StyleGAN latent z, which StyleGAN turns into a realistic image.

GANs were hard to train; autoencoders are old and general

GANs and StyleGANs were a highly active research area. They generate highly realistic images, and researchers were partly able to interpret their latent variables. Their minimax formulation made them very difficult to train, and researchers moved on to other approaches.

The autoencoder is an old idea from the 1980s. At that time it was a neural network alternative to principal component analysis (PCA). It is now seen as an extremely powerful and general concept, and the popular diffusion model is a special case of it.

The three families learn differently. A GAN trains a generator against a discriminator. A variational autoencoder (VAE) trains an encoder and a decoder together, by maximizing a lower bound on the probability of the data. A diffusion model adds Gaussian noise to the data in many small steps and learns to reverse them. In the figure, the GAN's D sees a generated image $\mathbf{x}'$ and a real image $\mathbf{x}$ and outputs 0 (fake) or 1 (real); the diffusion states end in $\mathbf{z}$, which is $\mathbf{x}_T$.

GANs were hard to train; autoencoders are old and general

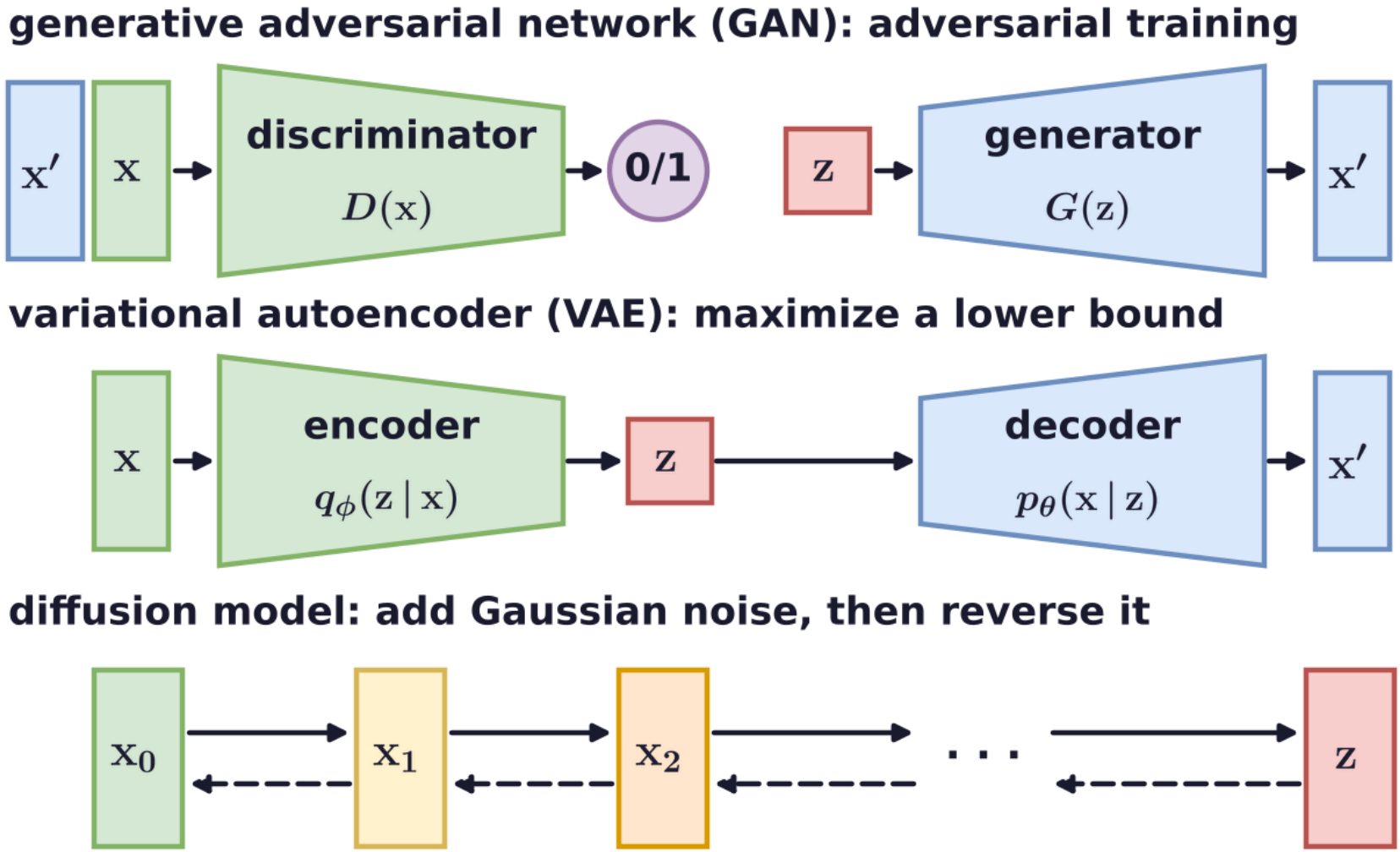


Figure 13: The three generative models (after Weng, 2021, from the original slides). Dashed arrows: the learned reverse steps; the colors of $\mathbf{x_1}$ and $\mathbf{x_2}$ only tell the states apart.

An autoencoder squeezes the input through a bottleneck

An autoencoder squeezes its input through a bottleneck and reconstructs it. The *encoder* g_ϕ transforms the high-dimensional input $\mathbf{x}$ (an image of P pixels) into a low-dimensional latent $\mathbf{z} = g_\phi(\mathbf{x})$ of $d \ll P$ numbers; the *decoder* f_θ recovers the data from it, $\mathbf{x}' = f_\theta(\mathbf{z})$; ideally $\mathbf{x}' = \mathbf{x}$. With $d < P$ the network cannot copy its input: it must find the few numbers that describe its training images best.

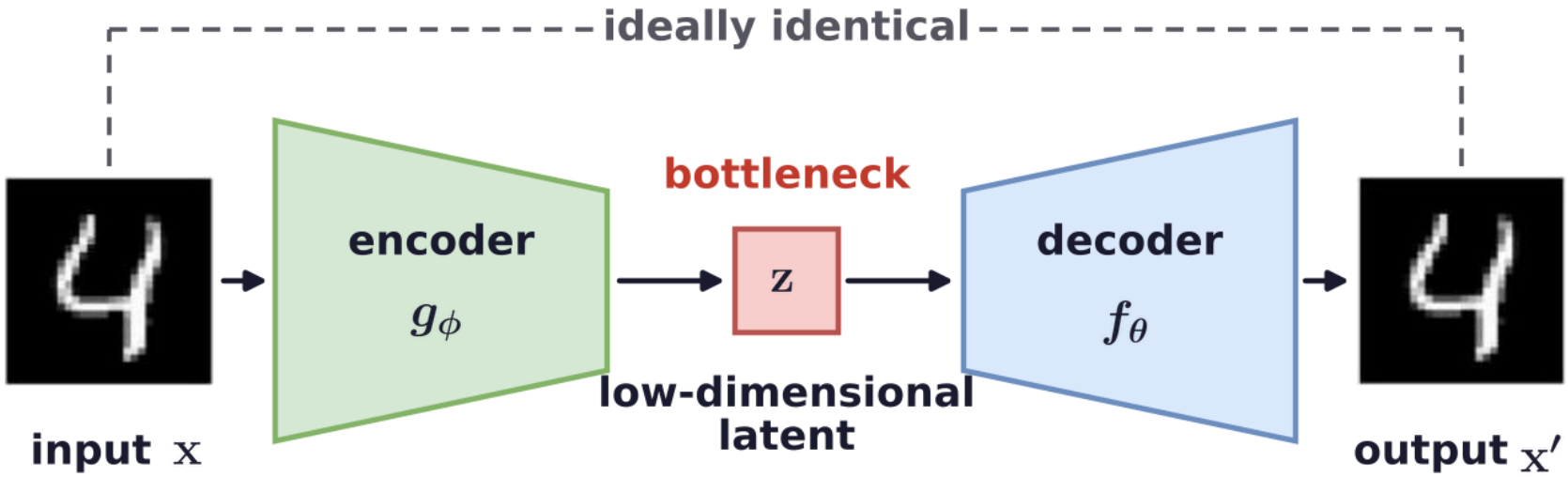


Figure 14: An autoencoder (after Weng, 2018, from the original slides): a 28×28 digit image is encoded into a short $\mathbf{z}$ and decoded.

An autoencoder minimizes the reconstruction loss

The encoder and the decoder are trained together on n training images $\mathbf{x}^{(i)}$ by gradient descent on the squared reconstruction error:

$$L_{\mathrm{AE}}(\theta, \phi) = \frac{1}{n} \sum_{i=1}^{n} \left\| \mathbf{x}^{(i)} - f_\theta\left(g_\phi(\mathbf{x}^{(i)})\right) \right\|^2.$$

The hope is a code that loses little: a latent that preserves most of the information of the input. This is dimensionality reduction, which links the autoencoder to PCA.

Take linear maps, $g(\mathbf{x}) = A\mathbf{x}$ and $f(\mathbf{z}) = B\mathbf{z}$, with A of size $d \times P$, and data with mean zero and covariance Σ. The loss is then the error of the rank-d map BA. Baldi and Hornik (1989) show that at the optimum BA is the orthogonal projection onto the span of the d top eigenvectors of Σ, the principal subspace, and that every local minimum is the global one. A linear autoencoder therefore finds PCA; nonlinear encoders and decoders can follow curved sets of images.

A variational autoencoder makes the latent a distribution

The autoencoder's latent space is not necessarily organized so that it can be sampled to generate images: nothing in its training makes sampling easy. A latent drawn at random may fall between the codes of the training images, where the decoder was never trained, and give an image of nothing.

Variational autoencoders (VAEs; Kingma and Welling, 2014) are autoencoders whose latent space is regularized to follow a specific, structured distribution, the prior $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$. The latent representation of $\mathbf{x}$ is now a distribution $q_\phi(\mathbf{z} \mid \mathbf{x})$ rather than a single point: the encoder outputs a mean $\boldsymbol{\mu}_\phi(\mathbf{x})$ and a standard deviation $\boldsymbol{\sigma}_\phi(\mathbf{x})$, one per latent dimension, and $q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}\big(\boldsymbol{\mu}_\phi(\mathbf{x}), \operatorname{diag} \boldsymbol{\sigma}^2_\phi(\mathbf{x})\big)$. The decoder turns a sample $\mathbf{z}$ into an image.

Training keeps every $q_\phi(\mathbf{z} \mid \mathbf{x})$ close to the prior, so the codes fill the space that the prior covers. To generate, draw $\mathbf{z}$ from $\mathcal{N}(\mathbf{0}, \mathbf{I})$ and decode it.

A variational autoencoder makes the latent a distribution

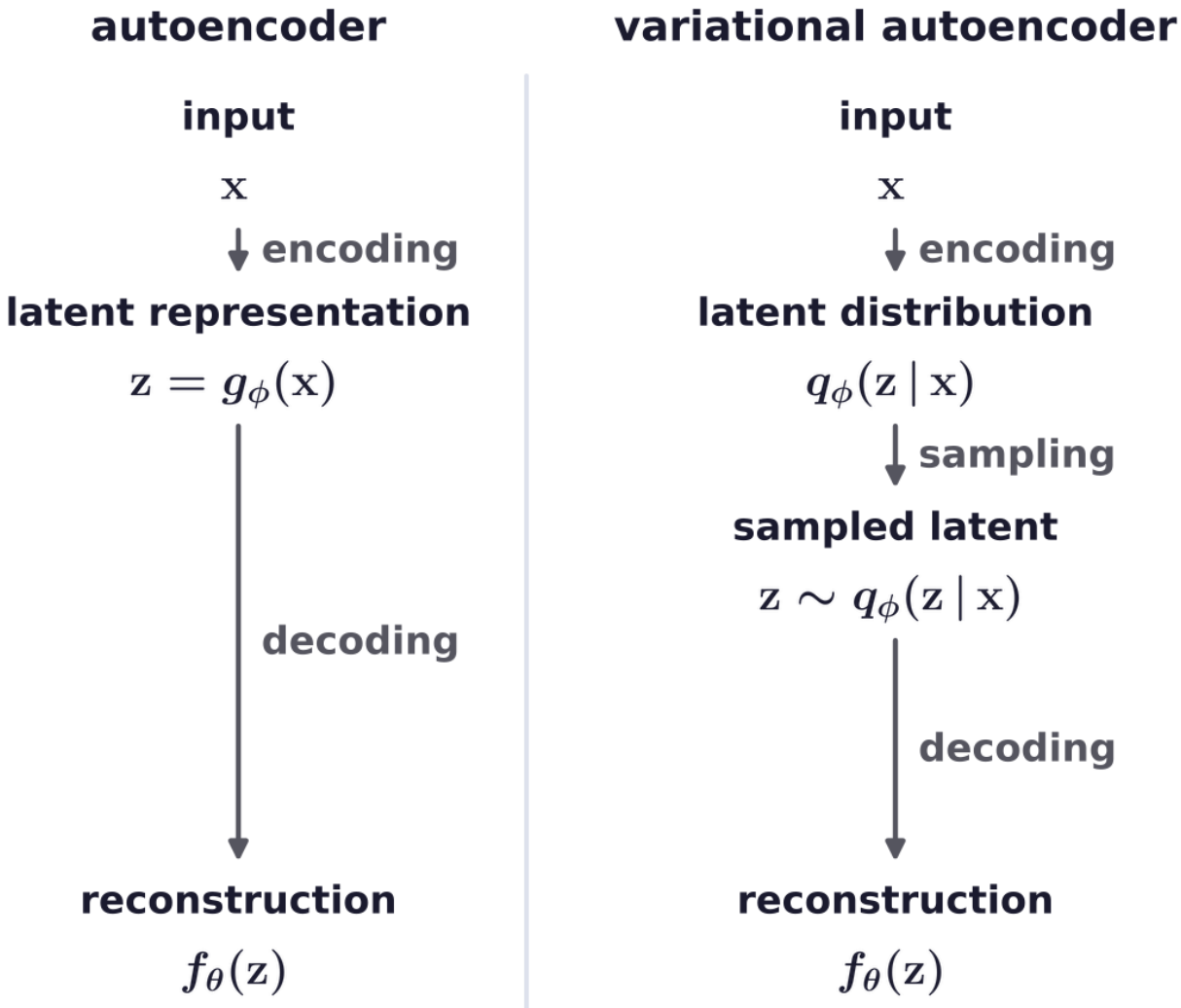


Figure 15: Autoencoder and variational autoencoder (the diagram of the original slides, drawn top to bottom): the autoencoder maps $\mathbf{x}$ to one point $\mathbf{z}$; the VAE maps it to a distribution, from which $\mathbf{z}$ is sampled and decoded.

The VAE loss is a variational bound, as in mean field theory

The decoder defines $p_\theta(\mathbf{x}, \mathbf{z}) = p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})$ and $p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x}, \mathbf{z})\, d\mathbf{z}$, an integral that cannot be computed; so the posterior $p_\theta(\mathbf{z} \mid \mathbf{x}) = p_\theta(\mathbf{x}, \mathbf{z}) / p_\theta(\mathbf{x})$ cannot be computed either. Write $\log p_\theta(\mathbf{x}) = \log p_\theta(\mathbf{x}, \mathbf{z}) - \log p_\theta(\mathbf{z} \mid \mathbf{x})$, add and subtract $\log q_\phi(\mathbf{z} \mid \mathbf{x})$, and average over q_ϕ:

$$\log p_\theta(\mathbf{x}) = \mathbb{E}_{q_\phi}\big[\log p_\theta(\mathbf{x}, \mathbf{z}) - \log q_\phi(\mathbf{z} \mid \mathbf{x})\big] + D_{\mathrm{KL}}\big(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p_\theta(\mathbf{z} \mid \mathbf{x})\big).$$

The VAE loss is minus the first term:

$$\begin{aligned} L_{\mathrm{VAE}}(\theta, \phi) &= -\log p_\theta(\mathbf{x}) + D_{\mathrm{KL}}\big(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p_\theta(\mathbf{z} \mid \mathbf{x})\big) \\ &= -\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z} \mid \mathbf{x})} \log p_\theta(\mathbf{x} \mid \mathbf{z}) + D_{\mathrm{KL}}\big(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z})\big). \end{aligned}$$

The KL divergence is at least 0, so L_{VAE} is an upper bound on $-\log p_\theta(\mathbf{x})$; minimizing it raises the probability of the data and pulls q_ϕ toward the true posterior.

The VAE loss is a variational bound, as in mean field theory

The second line of L_{VAE} has two terms: a reconstruction term (decode a sample of q_ϕ and score it against $\mathbf{x}$), and a KL term that pulls $q_\phi(\mathbf{z} \mid \mathbf{x})$ toward the prior. For Gaussians it has a closed form, $\frac{1}{2}\sum_j(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1)$.

The idea is that of mean field theory: approximate an intractable distribution by a simpler, factorized one, chosen to minimize a free energy, which is L_{VAE}. Here q_ϕ factorizes over the latent dimensions, and one encoder gives its parameters for every image.

The reconstruction term needs gradients of an expectation over q_ϕ with respect to ϕ. The reparameterization writes the sample as

$$\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),$$

with $\odot$ the elementwise product. The randomness is now in $\boldsymbol{\varepsilon}$, which does not depend on ϕ, so the gradient passes through the sampling step and one sample of $\boldsymbol{\varepsilon}$ per image is enough.

Diffusion models conditioned on text create realistic images

Diffusion models conditioned on text are able to create complex and realistic images. The prize-winning image of the slide, "Théâtre D'opéra Spatial", was created almost entirely by such a model: Jason M. Allen made it with Midjourney, and it won the digital-arts category of the 2022 Colorado State Fair.

These models take advantage of the huge advances of large language models. A large language model is auto-regressive: it predicts a text one word (token) at a time, each from the words before it. Trained on very large amounts of text, such models learn representations of sentences that capture their meaning. A text-to-image model encodes the prompt with a large pretrained Transformer text encoder (T5 for Imagen, CLIP for Stable Diffusion); Saharia et al. (2022) found that a larger text encoder improves the images more than a larger image model does.

Diffusion models conditioned on text create realistic images



Figure 16: "Théâtre D'opéra Spatial" (J. M. Allen, made with Midjourney; first prize in digital arts, Colorado State Fair 2022), from the original slides.

A diffusion model is an autoencoder with a noise encoder

A diffusion model (Sohl-Dickstein et al., 2015; Ho et al., 2020) is an autoencoder whose encoder adds noise. The *forward diffusion process* takes the input image $\mathbf{x}_0$ and outputs latent variables $\mathbf{x}_1, \ldots, \mathbf{x}_T$, each a slightly noisier copy of the one before; $\mathbf{x}_T$ is nearly pure noise. The *reverse diffusion process* takes the latent variables back to an image, one step $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ at a time.

Two things differ from an ordinary autoencoder: the latents have the size of the image, and the encoder is fixed (adding noise has nothing to learn), so only the decoder is learned. The true reverse step $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ is unknown, since it depends on the distribution of the images; training uses a variational bound, as for the VAE, with $\mathbf{x}_1, \ldots, \mathbf{x}_T$ as the latent variables.

A diffusion model is an autoencoder with a noise encoder

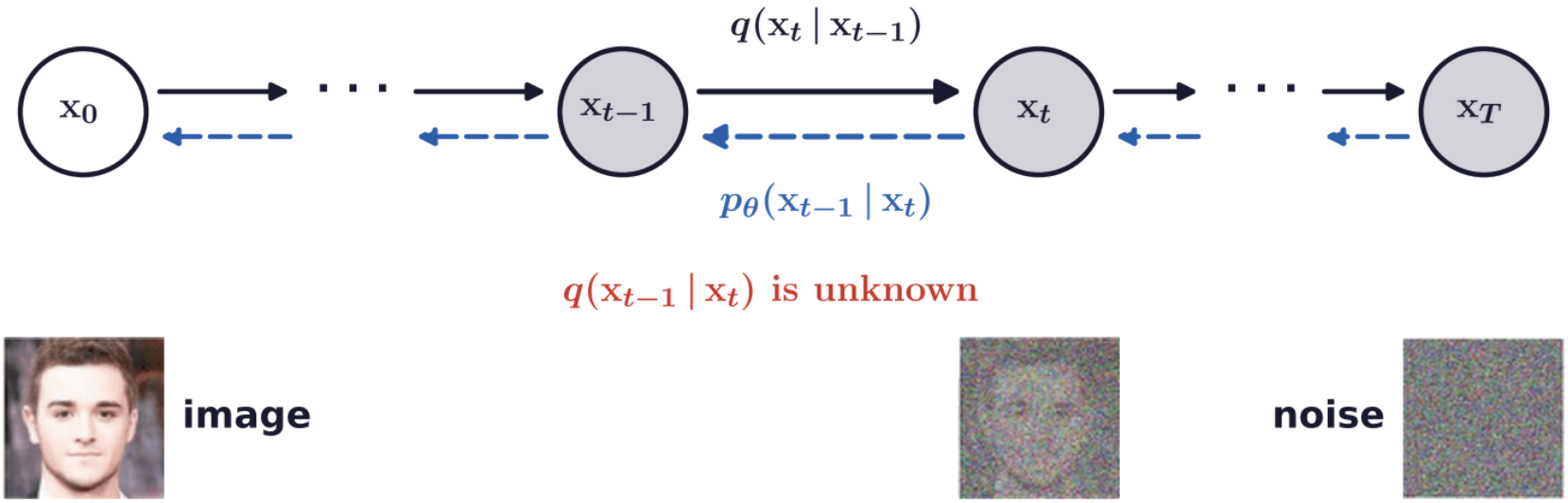


Figure 17: The diffusion chain (after Ho et al., 2020, from the original slides): forward steps q (black), learned reverse steps p_θ (blue, dashed), latent variables in grey.

Each forward step adds a little Gaussian noise

Each forward step shrinks the image a little and adds a little Gaussian noise (Ho et al., 2020, eq. 2):

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\big(\mathbf{x}_t; \sqrt{1-\beta_t}\,\mathbf{x}_{t-1}, \beta_t \mathbf{I}\big),$$

that is, $\mathbf{x}_t = \sqrt{1-\beta_t}\,\mathbf{x}_{t-1} + \sqrt{\beta_t}\,\boldsymbol{\varepsilon}_t$. Here $\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \Sigma)$ is the Gaussian density in $\mathbf{x}$ with mean $\boldsymbol{\mu}$ and covariance Σ, $\mathbf{I}$ the identity matrix, and $\boldsymbol{\varepsilon}_t$ a fresh standard Gaussian vector. The variance β_t is small and set by a schedule: Ho et al. increase it linearly from 10^{-4} to 0.02 over $T = 1000$ steps. The scaling by $\sqrt{1-\beta_t}$ keeps the variance at 1 when it starts at 1: $(1-\beta_t)\cdot 1 + \beta_t = 1$.

Since every step is linear with Gaussian noise, $\mathbf{x}_t$ is Gaussian given $\mathbf{x}_0$, and the t steps can be done in one.

Each forward step adds a little Gaussian noise

Write $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$. Then (Ho et al., eq. 4)

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\,\boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

By induction: if $\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}}\,\boldsymbol{\varepsilon}'$, then $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{\alpha_t(1 - \bar{\alpha}_{t-1})}\,\boldsymbol{\varepsilon}' + \sqrt{\beta_t}\,\boldsymbol{\varepsilon}_t$, and the two independent noise terms add to one Gaussian of variance $\alpha_t - \bar{\alpha}_t + 1 - \alpha_t = 1 - \bar{\alpha}_t$.

The weight $\sqrt{\bar{\alpha}_t}$ of $\mathbf{x}_0$ falls from 1 toward 0 as t grows. With Ho et al.'s schedule it is 0.95 at $t = 100$, 0.72 at $t = 250$, 0.28 at $t = 500$ and 0.0064 at $t = 1000$. At $t = T$ almost nothing of $\mathbf{x}_0$ is left: $\mathbf{x}_T$ is close to pure Gaussian noise, which is why sampling can start from $\mathcal{N}(\mathbf{0}, \mathbf{I})$.

The network learns to predict the added noise

The reverse step is Gaussian too, with a learned mean and a fixed variance (Ho et al., eq. 1 and Sec. 3.2: $\sigma_t^2 = \beta_t$, or the posterior variance $\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$, with similar results):

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}\big(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t),\, \sigma_t^2 \mathbf{I}\big).$$

The true reverse step $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ is unknown, but it becomes tractable once $\mathbf{x}_0$ is known. By Bayes' rule on the Gaussians, $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$ is Gaussian with mean

$$\tilde{\boldsymbol{\mu}}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\,\mathbf{x}_0 + \frac{\sqrt{\alpha_t}\,(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\,\mathbf{x}_t = \frac{1}{\sqrt{\alpha_t}}\Big(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\varepsilon}\Big),$$

where the second form substitutes $\mathbf{x}_0 = (\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\varepsilon})/\sqrt{\bar{\alpha}_t}$.

The network learns to predict the added noise

The Gaussian comes from Bayes' rule, $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \propto q(\mathbf{x}_t \mid \mathbf{x}_{t-1})\, q(\mathbf{x}_{t-1} \mid \mathbf{x}_0)$: both factors are Gaussian in $\mathbf{x}_{t-1}$, so their product is too, and completing the square gives $\tilde{\boldsymbol{\mu}}_t$ and the variance $\tilde{\beta}_t$.

With $\mathbf{x}_{1:T}$ as the latent variables, the bound of the VAE becomes (Ho et al., eq. 5)

$$L = D_{\mathrm{KL}}\big(q(\mathbf{x}_T \mid \mathbf{x}_0) \,\|\, p(\mathbf{x}_T)\big) - \log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1) + \sum_{t>1} D_{\mathrm{KL}}\big(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \,\|\, p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)\big).$$

The first term has no learned parameters. In each term of the sum both distributions are Gaussian with the same fixed variance, so the KL divergence is $\|\tilde{\boldsymbol{\mu}}_t - \boldsymbol{\mu}_\theta(\mathbf{x}_t, t)\|^2/(2\sigma_t^2)$ plus a constant: each term asks $\boldsymbol{\mu}_\theta$ to match $\tilde{\boldsymbol{\mu}}_t$.

The network learns to predict the added noise

Since $\mathbf{x}_t$ is an input, only the noise ε in $\tilde{\boldsymbol{\mu}}_t$ is unknown. Ho et al. therefore let a network $\varepsilon_\theta(\mathbf{x}_t, t)$ of the noisy image and the timestep predict it, and set $\boldsymbol{\mu}_\theta = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(\mathbf{x}_t, t)\right)$. Each KL term then becomes a weighted squared error between ε and ε_θ. Dropping the weights gives the training loss (eq. 14):

$$L_{\text{simple}} = \mathbb{E}_{t,\mathbf{x_0},\varepsilon}\left[\left\|\varepsilon - \varepsilon_\theta\left(\sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon,\ t\right)\right\|^2\right].$$

This is a simplified, reweighted form of the bound. Training is short to state: take a training image $\mathbf{x}_0$, a timestep t drawn uniformly from 1 to T and noise ε; form $\mathbf{x}_t$ in one step; and ask the network to predict ε. Predicting the noise gives the mean of the reverse step.

The denoising network is a U-Net

The network ε_θ is a variant of the U-Net architecture (Ronneberger et al., 2015), first designed to segment biomedical images. Its input and output are images of the same size, which suits a network that predicts the noise of every pixel.

- The *encoder path* applies convolutions and downsampling, so that deeper features cover larger parts of the image.
- The *decoder path* upsamples back to the input size.
- *Skip connections* copy the features of the encoder path to the decoder path at the same resolution, where they are concatenated with the upsampled features; fine detail does not have to pass the bottom of the U.

The diffusion U-Net of Ho et al. (2020) is built from residual blocks, with self-attention (every position of the feature map is updated with a weighted average of all positions) at the 16×16 resolution. It also takes the timestep t: t is turned into a vector, the *time embedding*, and added to the features of every residual block, so that one network serves all noise levels. The details are out of the scope of the course.

Sampling denoises Gaussian noise step by step

To generate an image, sample the latent variables: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, with $T = 1000$. Iterative sampling then generates an image (Ho et al., Algorithm 2): for $t = T, \ldots, 1$,

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{1-\beta_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\varepsilon_\theta(\mathbf{x}_t, t)\right) + \sigma_t \mathbf{n}, \qquad \mathbf{n} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),$$

with $\mathbf{n} = \mathbf{0}$ at the last step. Each step subtracts the share of the predicted noise that belongs to step t, rescales, and adds a little fresh noise.



Figure 18: Sampling (from the original slides): Gaussian noise $\mathbf{x}_T$, denoised step by step into a dog. Each arrow is one step with the U-Net.

Stable Diffusion performs diffusion in a latent space

Diffusion on the pixels of a large image is slow: every one of the thousand steps runs the U-Net on the full image. Stable Diffusion, the latent diffusion model of Rombach et al. (2022), performs diffusion in a latent space instead.

- An autoencoder is trained first, once: the encoder E maps an image $\mathbf{x}_0$ to a latent $\mathbf{z}_0$ that is 8 times smaller per side (512×512 pixels become 64×64 positions with 4 channels), and the decoder D maps it back. Here D is a decoder, unrelated to the discriminator of a GAN.
- The forward and reverse diffusion processes run on the latents $\mathbf{z}_0, \ldots, \mathbf{z}_T$, with a denoising U-Net, exactly as on images.
- To generate, the reverse process turns noise $\mathbf{z}_T$ into $\mathbf{z}_0$, and the decoder turns $\mathbf{z}_0$ into the image $\mathbf{x}'_0$.

The autoencoder removes the imperceptible detail of the image, so diffusion in the small latent space is much cheaper than in pixel space and spends its effort on the content.

Stable Diffusion performs diffusion in a latent space

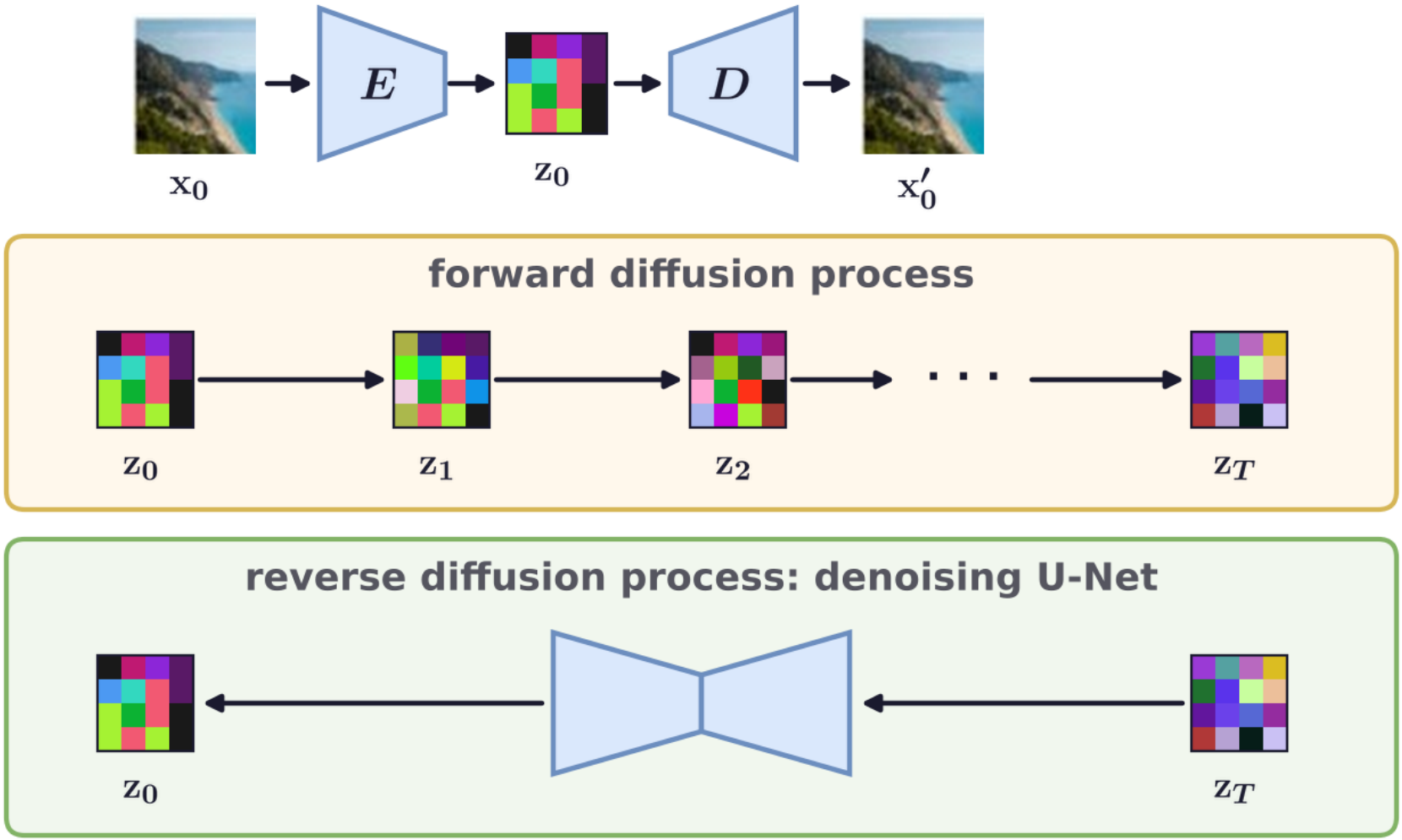


Figure 19: Stable Diffusion (Rombach et al., 2022, from the original slides): the autoencoder with encoder E and decoder D (top); the forward and reverse diffusion processes on the latent (the colored grids are latent arrays).

Cross-attention conditions Stable Diffusion on text

Stable Diffusion is conditioned on text by a cross-attentional mechanism, the attention of Transformers (Vaswani et al., 2017); this enables prompting on text. A text encoder τ_θ, a Transformer, turns the prompt y (unrelated to the style y of StyleGAN) into a sequence of token embeddings, one vector per word piece. Inside the U-Net, at several layers,

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\Big(\frac{QK^{\mathsf{T}}}{\sqrt{d}}\Big)V,$$

with $Q = W_Q\,\varphi(\mathbf{z}_t)$, $K = W_K\,\tau_\theta(y)$ and $V = W_V\,\tau_\theta(y)$. The features $\varphi(\mathbf{z}_t)$ of the noisy latent give one query per position; the token embeddings give the keys and the values; W_Q, W_K and W_V are learned matrices and d the length of the keys. The softmax turns the scores of each query into weights over the tokens, so each position of the latent collects the values of the words most relevant to it, and the result is added to its features. A "horse" in the prompt thus steers the positions where a horse is being formed.

Cross-attention conditions Stable Diffusion on text

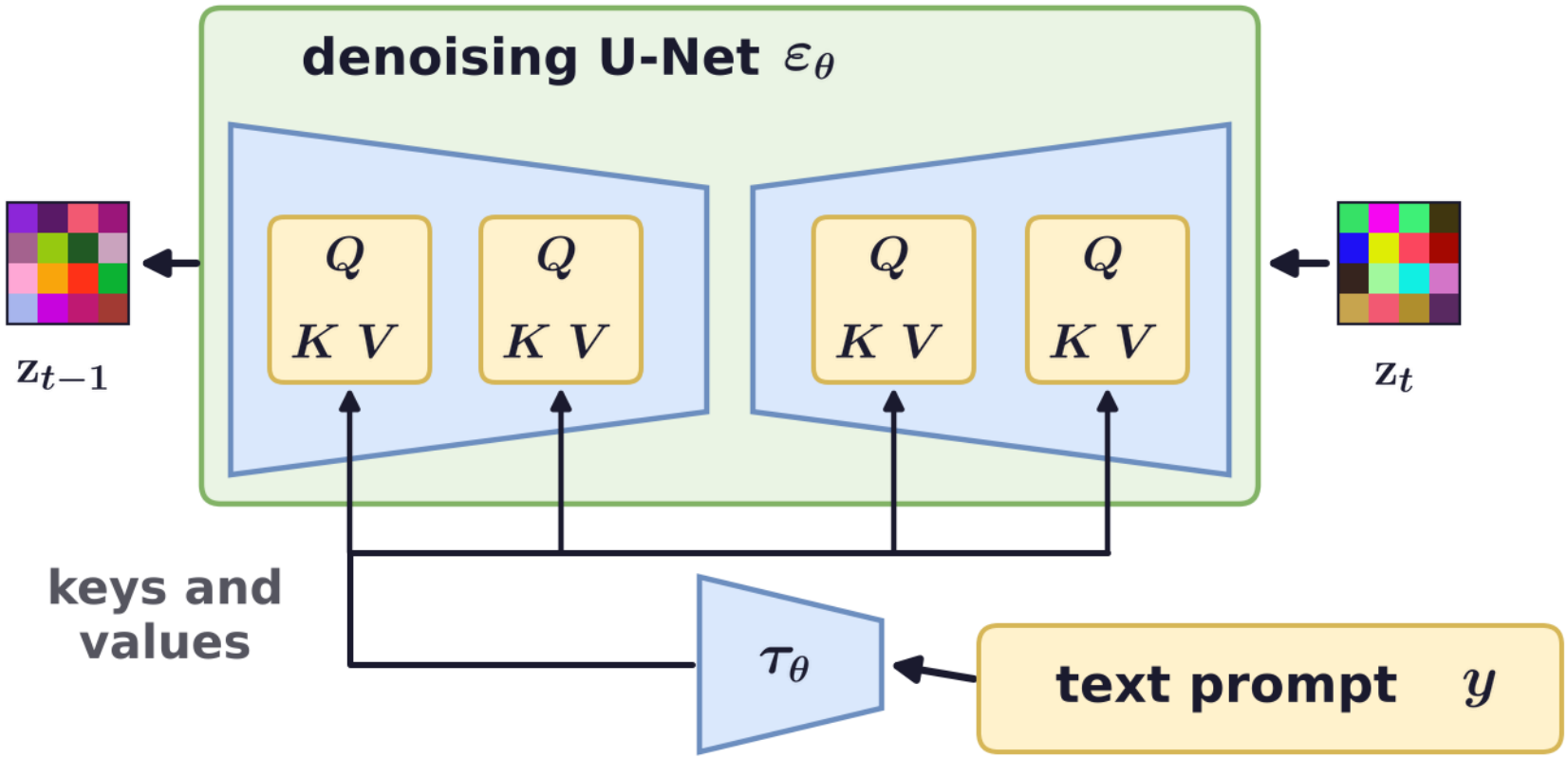


Figure 20: Text conditioning (after Rombach et al., 2022, Figure 3, from the original slides): the prompt y is encoded by τ_θ; its keys and values enter the cross-attention blocks (Q, K, V) of the denoising U-Net ε_θ, which turns $\mathbf{z_t}$ into $\mathbf{z_{t-1}}$.

Text prompts control the generated images

The prompts combine objects and actions that hardly occur together in photographs, and the models compose them into one scene. The images come from other text-to-image diffusion models: (a) is DALL-E 2's example, (b) and (c) are from Imagen (Saharia et al., 2022).

(a) An astronaut riding a horse in photorealistic style.

(b) A chrome-plated duck with a golden beak arguing with an angry turtle in a forest.

(c) A cute corgi lives in a house made out of sushi.

(d) A dog looking curiously in the mirror, seeing a cat.



Figure 21: Images generated from the four prompts (from the original slides): (a) DALL-E 2, (b) and (c) Imagen; the model behind (d) could not be confirmed.

Diffusion models lack conditioning on the 3D world state

Diffusion models (DMs) can generate a very rich variety of realistic images controlled by text prompts, and they can be extended to generate videos. This is a research industry.

From the computer vision perspective, they are lacking as generative models. Bayesian analysis by synthesis wants a generative model $P(I \mid W)$ conditioned on the state W of the 3D world (the shapes, materials and lighting of the scene, and the camera), so that an image can be explained by inverting it: $P(W \mid I) \propto P(I \mid W)P(W)$. A DM generates an image from noise and a prompt, and the 3D state never appears as a variable.

DMs do have implicit representations of the 3D world: their internal features carry information about depth and shape. These representations are partially understood, and they can be exploited to estimate 3D properties of images. Can implicit representations emerge within DMs? Some people are betting a lot of money that they will. Alan Yuille is not convinced.

Summary

- Deep generative models learn from big data; they are realistic but hard to interpret.
- A GAN trains a generator against a discriminator in a minimax game: hard to train.
- StyleGAN maps z to a latent w that sets the style at every level; $\mathcal{W}$ is more disentangled than $\mathcal{Z}$.
- GAN inversion finds the latent code of a real image: editing, colorization, denoising, inpainting.
- An autoencoder reconstructs its input through a bottleneck; a VAE makes the latent a distribution.
- A diffusion model is an autoencoder whose encoder adds noise; a U-Net learns to remove it.
- Stable Diffusion denoises in a latent space and is conditioned on text by cross-attention.
- Diffusion models lack conditioning on the 3D world state that analysis by synthesis needs.

References

[1] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville and Y. Bengio. Generative adversarial nets. *NeurIPS*, 2672–2680, 2014.

[2] D. P. Kingma and M. Welling. Auto-encoding variational Bayes. *ICLR*, 2014.

[3] J. Ho, A. Jain and P. Abbeel. Denoising diffusion probabilistic models. *NeurIPS*, 6840–6851, 2020.

[4] R. Rombach, A. Blattmann, D. Lorenz, P. Esser and B. Ommer. High-resolution image synthesis with latent diffusion models. *CVPR*, 10684–10695, 2022.

[5] L. Weng. From autoencoder to beta-VAE. *lilianweng.github.io*, 2018.

[6] L. Weng. What are diffusion models? *lilianweng.github.io*, 2021.

[7] A. Radford, L. Metz and S. Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *ICLR*, 2016.

[8] T. Karras, T. Aila, S. Laine and J. Lehtinen. Progressive growing of GANs for improved quality, stability, and variation. *ICLR*, 2018.

References

[9] T. Karras, S. Laine and T. Aila. A style-based generator architecture for generative adversarial networks. *CVPR*, 4401–4410, 2019.

[10] T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen and T. Aila. Analyzing and improving the image quality of StyleGAN. *CVPR*, 8110–8119, 2020.

[11] A. Brock, J. Donahue and K. Simonyan. Large scale GAN training for high fidelity natural image synthesis. *ICLR*, 2019.

[12] T. Karras, M. Aittala, J. Hellsten, S. Laine, J. Lehtinen and T. Aila. Training generative adversarial networks with limited data. *NeurIPS*, 12104–12114, 2020.

[13] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi and R. Ng. NeRF: representing scenes as neural radiance fields for view synthesis. *ECCV*, 405–421, 2020.

[14] OpenAI. DALL-E 2. *https://openai.com/dall-e-2/*, 2022.

[15] C. Saharia, W. Chan, S. Saxena et al. Photorealistic text-to-image diffusion models with deep language understanding. *NeurIPS*, 36479–36494, 2022.

[16] I. Goodfellow. NIPS 2016 tutorial: generative adversarial networks. *arXiv:1701.00160*, 2016.

References

[17] Z. Wu, D. Lischinski and E. Shechtman. StyleSpace analysis: disentangled controls for StyleGAN image generation. *CVPR*, 12863–12872, 2021.

[18] J. Gu, Y. Shen and B. Zhou. Image processing using multi-code GAN prior. *CVPR*, 3012–3021, 2020.

[19] J. Zhu, Y. Shen, D. Zhao and B. Zhou. In-domain GAN inversion for real image editing. *ECCV*, 592–608, 2020.

[20] Y. Zhang, W. Chen, H. Ling, J. Gao, Y. Zhang, A. Torralba and S. Fidler. Image GANs meet differentiable rendering for inverse graphics and interpretable 3D neural rendering. *ICLR*, 2021.

[21] P. Baldi and K. Hornik. Neural networks and principal component analysis: learning from examples without local minima. *Neural Networks*, 2(1):53–58, 1989.

[22] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan and S. Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. *ICML*, 2256–2265, 2015.

[23] O. Ronneberger, P. Fischer and T. Brox. U-Net: convolutional networks for biomedical image segmentation. *MICCAI*, 234–241, 2015.

[24] A. Vaswani, N. Shazeer, N. Parmar et al. Attention is all you need. *NeurIPS*, 5998–6008, 2017.